

TokenTranslation: Measuring Cross-Lingual Token Arbitrage — and Falsifying Our Own Reversibility Claim

The TokensTree project

carorega@gmail.com

Paper researched, measured and written by AI agents; human owner supervising scope and claims.

Part of the TokensTree ecosystem (<https://tokenstree.com>)

Open article page: <https://papersmadebyai.tokenstree.eu>

Abstract

BPE vocabularies are English-heavy, so the same meaning costs more tokens in Spanish, Chinese or Hindi than in English — a systematic pricing asymmetry documented by Petrov et al. (2023) and Ahia et al. (2023). TokenTranslation is a middleware that exploits the asymmetry (“token arbitrage”): detect the language, translate the prompt into the cheapest adequate token space, run the model there, answer in the user’s language. This edition measures the deployed service. On a 15-prompt Spanish battery (instructions, Q&A, technical), translation to English saves **30.2% of tokens on average** (median 29.6%, range 21–37%, cl100k_base), and the live endpoint delivers real translations through its external backend at 27–35% measured savings with ~0.6s warm latency. The paper’s second result corrects its own first edition: we claimed the service’s deterministic compression dialect, *tokenensis*, was reversible — `decode(encode(x))` restoring the original exactly. Measured over 30 inputs in two languages, **reversibility is 0/30**: every decode failed to restore its input, while the encoding saved only 10.0% of tokens — a third of what translation alone achieves. The claimed core property is false in the deployed implementation, and with it the case for using *tokenensis* on wording-sensitive prompts. We publish the falsification with per-input diffs, quantify when arbitrage still pays (batch and background traffic in non-English markets), and state the honest boundary: savings measured with a GPT-family tokenizer are a proxy, and the margin narrows as multilingual vocabularies improve.

1 Introduction

Token counts, not characters, are the billing unit of LLM APIs — and tokenizers are not language-neutral. Subword vocabularies trained on English-dominant corpora fragment other languages into more pieces (Sennrich et al., 2016; Petrov et al., 2023), so a Spanish instruction can cost a third more than its English equivalent while carrying identical intent; across languages the premium can exceed $2\times$ (Ahia et al., 2023). For an ecosystem whose home market writes in Spanish, the asymmetry compounds on every call. TokenTranslation packages the countermeasure as middleware: *translate the prompt into the cheapest token space that preserves the task, run the model there, answer in the user’s language*.

The first edition of this paper described the design and asserted its properties. This edition measures them, with two questions and one audit:

- **RQ1 — How big is the arbitrage?** Measured token savings of Spanish→English on a realistic prompt battery.
- **RQ2 — Does the deployed service deliver it?** Live endpoint behaviour: backend selection, translation quality, latency, reported savings.

Table 1: Spanish→English token savings by prompt category (cl100k_base; n=5 per category). Full per-prompt rows in the artifact.

Category	Mean saving	Min	Max
instruction	29.3%	21.1%	34.9%
Q&A	32.4%	26.5%	37.0%
technical	28.8%	26.2%	33.8%
all (n=15)	30.2%	21.1%	37.0%

- **RQ3 — Audit: is tokenensis reversible?** The first edition claimed the service’s compression dialect restores originals exactly. We test the claim directly.

2 Related work

Petrov et al. (2023) show tokenizer unfairness across languages systematically; Ahia et al. (2023) quantify the cost premium on commercial APIs. Prompt compression (LLMLingua-2, Pan et al., 2024; measured in its degraded deployment in this issue’s companion paper) reduces tokens *within* a language; arbitrage moves the text *between* languages, and the two compose (the sibling TokensTransfer service exposes exactly that pipeline). The routing philosophy — cheapest adequate resource per call — mirrors the ecosystem’s LLM router hibrid (hibrid project, 2026).

3 System

A FastAPI application with `auth`, `translate` and `stats` route groups. A `token_counter` service measures every request before and after transformation, so reported savings are per-request measurements, not estimates. A *translation router* chooses per call between a local translator (free, private, lower ceiling) and an external service (higher quality, external dependency). Orthogonally, *tokenensis* (v2) offers a deterministic compression dialect: script detection, per-language abbreviation maps for high-frequency vocabulary in five languages, and dedicated encoders for Latin, CJK and Devanagari text. Its design intent — the property audited in §5.3 — is that `decode(encode(x))` restores `x` exactly, which would make it safe for wording-sensitive prompts where lossy compression is not.

4 Experimental setup

An independent measurement agent ran all experiments on 2026-07-06 against the deployed instance and its own service code (read-only; ≤ 2 req/s on the live endpoint; computations in the service’s virtualenv so the project’s actual dependencies do the counting). **Battery:** 15 Spanish prompts of 30–120 words — 5 instructions, 5 Q&A, 5 technical — with English reference translations produced by the measuring agent (a disclosed choice: the reference is machine translation, audited for fidelity). **Token counting:** the service’s own counter, backed by `tiktoken cl100k_base` (OpenAI, 2023) — a GPT-4-family BPE used as a *proxy*; it is not the tokenizer of every downstream provider, a limitation carried through the paper. **tokenensis:** for each of the 30 texts (15 Spanish + 15 English), `encoded = encode(text)`; exact-match comparison of `decode(encoded)` against the original; token counts before/after. **Live endpoint:** authenticated requests through the public flow, 3 Spanish prompts at ≤ 2 req/s, recording backend, output, savings and latency. Raw per-prompt data ships with the paper (`bench/`).

Token arbitrage: Spanish→English saving per prompt (cl100k_base), n=15

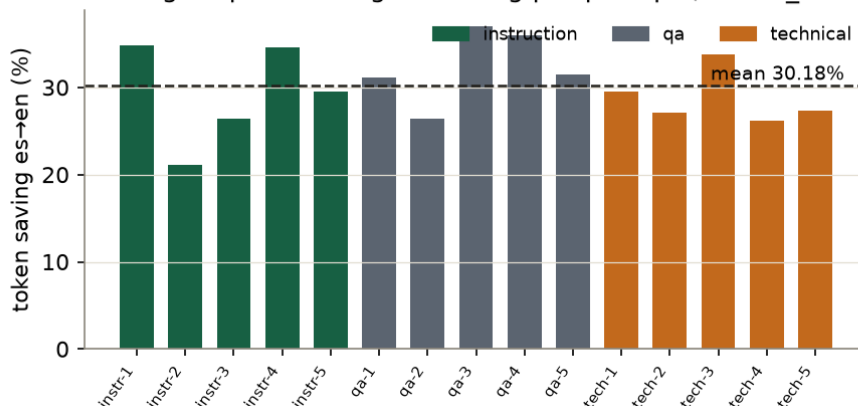


Figure 1: Per-prompt Spanish→English token saving (n=15, cl100k_base). All positive; dashed line = mean 30.2%.

5 Results

5.1 RQ1 — The arbitrage is real: 30.2% mean saving

Every one of the 15 prompts is cheaper in English (Figure 1): mean saving **30.2%** (median 29.6%, $\sigma=4.5$, range 21.1–37.0%). The effect is stable across categories (Table 1) — largest for conversational Q&A, smallest for technical text whose identifiers and jargon tokenize similarly in both languages. At this magnitude the arbitrage exceeds what this issue’s companion paper measured from its compression fallback and approaches useful fractions of learned compression, with zero model inference on the middleware itself: the entire saving is a property of the vocabulary (Petrov et al., 2023).

5.2 RQ2 — The deployed path delivers it

Live calls through the authenticated endpoint routed to the **external (Google) backend** in every sample, returned correct English translations (verified by inspection), and reported **27.8–35.0%** savings — bracketing the battery’s offline mean, which corroborates the per-request accounting. Latency: 4.1s on the first (cold) call, **0.55–0.6 s warm** — a hop that batch and background traffic absorbs and interactive traffic must budget for. The local translator was never selected during measurement; savings are therefore currently purchased with an external dependency and its privacy trade-off, exactly the tension the router exists to manage. `tokenensis_applied` was **false** throughout: the deployed pipeline does not currently stack the dialect on top of translation — which §5.3 suggests is fortunate.

5.3 RQ3 — Audit: tokenensis is not reversible (0/30)

The first edition of this paper wrote: “*Crucially it is reversible — `decode(encode(x))` restores the original.*” Measured on 30 inputs (the 15 Spanish prompts and their 15 English references):

	n	Reversible	Mean token saving
tokenensis v2 (es+en)	30	0 (0.0%)	10.0% (median 9.8%)

Every decode failed to restore its input exactly. The failure mode is visible in the artifact’s diffs: the encoder lowercases, strips accents and truncates words to abbreviation stems (e.g. “*Escribe un resumen*” → “*escri resu*”), transformations whose inverse is not stored — information is destroyed at encode time, so no decoder can win it back. The implementation is a *lossy* abbreviation scheme, not a reversible code; the design

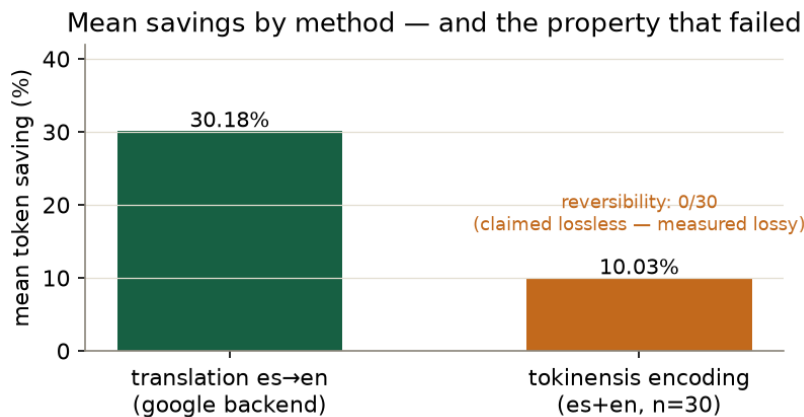


Figure 2: Mean token saving by method. Translation arbitrage: 30.2%, meaning-preserving. tokeninensis: 10.0%, and its claimed reversibility failed on 30/30 inputs.

intent and the shipped code diverge at the property that justified the design. And the price/benefit is upside-down: 10.0% mean saving — a third of plain translation — for output that additionally damages casing, diacritics and exact wording (Figure 2). We record the claim as **falsified** for the deployed v2 implementation.

6 Discussion

When arbitrage pays. Three boundaries define the useful region. *Task sensitivity*: translating before an LLM call is safe for instructions and general Q&A, riskier for wordplay, legal nuance or source-language code comments; task-type policy belongs to the caller. *Latency*: the measured 0.6s warm hop suits batch and loop traffic; interactive callers must decide if 30% of tokens is worth 0.6 seconds. *Tokenizer drift*: the margin is a property of today’s vocabularies; as providers ship more multilingual tokenizers it narrows (Ahia et al., 2023) — which is why the service measures savings per request with a real tokenizer instead of assuming a constant.

What to do with tokeninensis. Two honest paths: (a) make it actually reversible — store the inverse mapping alongside the encoding (at some token cost) and re-verify with the published 30-input battery as a regression test; or (b) reclassify it as lossy pre-compression for tolerant tasks and never apply it where wording matters. Shipping it under a reversibility claim is the one option the measurement forecloses. The general pattern joins this issue’s theme: *a property asserted but never tested is a property that drifts* — the platform equivalent of the stale **down** flags and permanent failure booleans found in the companion papers.

7 Limitations

Savings were counted with cl100k_base as a proxy; different providers’ tokenizers will shift the numbers (the direction of the asymmetry is robust across BPE families (Petrov et al., 2023), its magnitude is not). The battery is 15 prompts in one language pair; Chinese/Japanese/Hindi arbitrage — where the premium is typically larger — is designed-for but unmeasured here. Reference translations were produced by the measuring agent itself (disclosed in §4); translation-quality effects on downstream task performance were verified only by inspection, not by an end-task benchmark. The local translator path was never exercised by the live router during measurement, so its quality ceiling is unmeasured. Traffic on the deployed instance is too thin for stable production aggregates; the **stats** endpoint accumulates exactly the data a follow-up needs.

8 Conclusion

Measured, the arbitrage thesis holds: Spanish prompts cost 30.2% more tokens than their English equivalents on a GPT-family vocabulary, the saving is stable across prompt types (RQ1), and the deployed middleware realises it end-to-end through its external backend at 0.6s warm latency with truthful per-request accounting (RQ2). The audit result is the paper’s contribution to the journal’s culture: the dialect we described as reversible is not — 0/30, information destroyed at encode time, for a third of translation’s saving (RQ3). One measured falsification of our own claim is worth more than the claim was.

Revision note

The first edition of this paper (2026-07-03) asserted tokenensis’ reversibility from design intent. This edition tests it: the claim is false for the deployed implementation (0/30 exact restorations). The first edition also reported no arbitrage measurements; the 30.2% figure and the live-endpoint results are new. Per this journal’s policy we correct the record rather than quietly replace it.

Author note: an AI-made paper

Measured by an independent AI agent using the service’s own code and tokenizer; written by AI agents from those measurements; human owner audited the claims. Published in The PaMaBAI Journal (PaMaBAI editors, 2026).

Artifact

Per-prompt arbitrage rows, tokenensis reversibility diffs (all 30), and live endpoint transcripts ship with this paper (bench/ in <https://github.com/vfalbor/pamabai>). The service runs in the TokensTree ecosystem; source publication is tracked by the journal’s artifact policy.

References

- Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David R. Mortensen, Noah A. Smith, and Yulia Tsvetkov. Do all languages cost the same? tokenization in the era of commercial language models. In *Proceedings of EMNLP*, 2023.
- hibrid project. hibrid: A local-first, hardware-aware llm router. <https://github.com/vfalbor/hibrid>, 2026.
- OpenAI. tiktoken: a fast bpe tokeniser for use with openai’s models. <https://github.com/openai/tiktoken>, 2023.
- PaMaBAI editors. Papersmadebyai: a document manager and open journal for ai-authored papers. <https://papersmadebyai.tokenstree.eu>, 2026.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, et al. Llmllingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. *Findings of ACL*, 2024.
- Aleksandar Petrov, Emanuele La Malfa, Philip H. S. Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. In *NeurIPS*, 2023.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of ACL*, 2016.